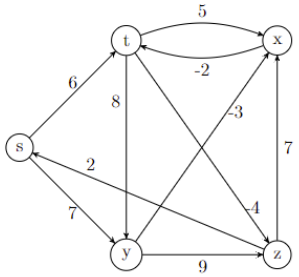
Evan Litzer
CS 311
4/26/2024

1.



Find the shortest path from the source vertex s to all other vertices. The table below will show the dictionary in each iteration regarding the calculated distance from the source vertex s.

- Firstly check for net negative cycles that would rule this entire formula out of the equation. There are none.
- In edge relaxation there will be V - 1 total iterations. So make a 2D array for the vertex dictionary, in that dict[iter][node #].

(0th Iter) S	T	Y	X	Z
0	∞	∞	∞	∞
(1st Iter) S	T	Y	X	Z
0	6	7	∞	∞
(2nd Iter) S	T	Y	X	Z
0	6	7	4	2
(3rd Iter) S	T	Y	X	Z
0	2	7	4	-2
(4th Iter) S	T	Y	X	Z
0	2	7	4	-2

S→S: 0 (Same node)
S→T: 2 (S→Y = 7, Y→X = -3, X→T = -2)
7 - 3 - 2 = 2
S→Y: 7 (S→Y = 7)
S→X: 4 (S→Y = 7, Y→X = -3)
7 - 3 = 4
S→Z: -2 (S→Y = 7, Y→X = -3, X→T = -2, T→X = -4)
7 - 3 - 2 - 4 = -2

2A. Recursive Definition

Base Case:

LCS(X, Y, a, b) = “” if a or b = 0.

A and B are tuples that keep track of what characters have been parsed through in the strings. If one of the tuples = 0, then it is an empty string.

LCS will automatically equal an empty string if either string length is empty because there is no chance of a shared common string of characters besides an empty string, which is equivalent to zero characters.

Therefore, return an empty string.

Recursive Case:

If X[a] == Y[b] for some character a in string X and some character b in String Y, then the character must be added to the LCS of the two strings. This character can be represented by either X[a] or Y[b], but they would be the same. Decrement both tickers a and b for both strings.

LCS(X, Y, a-1, b-1) + Y[b] (or X[a])

If X[a] != Y[b], the LCS of both of the strings will be the maximum length LCS not including either character from the current string. This is done recursively, as the maximum path must be found to continue iterating recursively.

LCS(X, Y, a, b) = max(LCS(X, Y, a, b-1).length, LCS(X, Y, a-1, b).length)

2B. Pseudocode

LCS(X, Y), where X and Y are inputted strings and the output is the longest common subsequence string that X and Y share:

```
    Let a and b be the tuple values that parse through strings.
    Initialize dict as an empty dictionary of empty strings. If needed, initialize every cell to
    an empty string.
    for a = 0 to X.length
        for b = 0 to Y.length
            if a == 0 || b == 0
                dict[a][b] == “”
            else if X[a-1] == Y[b-1]
                dict[a][b] = dict[a-1][b-1] + X[a-1] ;
            else
                if dict[a-1][b].length > if dict[a][b-1].length
                    dict[a][b] = dict[a-1][b]
                else
                    dict[a][b] = dict[a][b-1]
    return dict[a][b] ;
```

2C. RUNTIME

The runtime of this algorithm would be **O(a*b)** where a and b represent the length of the X and Y strings. Iterate over a*b cells in the dictionary when constructing the longest continuous sequence.

- Algorithm contains two for loops that iterate over the string-length amount of characters of each string. Outer loop runs a + 1 times and the inner loop runs b + 1 times, where a is length of X string and b is length of Y string.
- Constant operations per iteration of 2D array box in dictionary.
- So therefore, 1 * a * b = a * b runtime and algorithm uses dictionary and dynamic programming.